Cloud Disk Project

网盘新增功能流程说明

本文档面向项目交付、课程汇报和功能说明场景，整理本次新增功能的实现目标、设计思路和核心流程。

文档内容： 上传、预览、最近文件、搜索筛选排序、收藏、批量操作、回收站

排版形式： 纯 PDF 打印版

流程表示： ASCII 流程图，可直接阅读

一、文件上传流程

1. 功能目标

- 秒传
- 分片上传
- 断点续传
- 暂停 / 继续
- 失败重试
- 上传进度展示
- 剩余时间估算
- STS 临时授权上传

2. 设计思路

上传流程拆成三段：上传初始化、前端直传 OSS、上传完成确认。

这样可以减少后端带宽压力，并充分利用 OSS 的分片上传能力，同时保留前端任务进度、暂停继续、重试和秒传判断能力。

3. 流程图

```
+------------------------+
| 用户选择本地文件       |
+------------------------+
           |
           v
+------------------------+
| 前端计算文件 MD5       |
+------------------------+
           |
           v
+----------------------------+
| 调用 POST /file/upload/init |
+----------------------------+
           |
           v
+---------------------------------+
| 后端根据 hash + size 查询资源池  |
+---------------------------------+
      | 命中                            | 未命中
      v                                 v
+--------------------------+     +--------------------------+
| 直接写入 user_repository  |     | 创建 upload_session      |
+--------------------------+     +--------------------------+
      |                                 |
      v                                 v
```

```
+---------------------------+     +---------------------------+
| 返回 instant_hit = true   |     | 生成 objectKey            |
+---------------------------+     +---------------------------+
      |                                   |
      v                                   v
+---------------------------+     +---------------------------+
| 上传任务直接结束          |     | 调用 STS AssumeRole       |
+---------------------------+     +---------------------------+
                                          |
                                          v
                              +-------------------------------+
                              | 返回 STS + sessionIdentity    |
                              | + objectKey                   |
                              +-------------------------------+
                                          |
                                          v
                              +-------------------------------+
                              | 前端使用 ali-oss 直传 OSS     |
                              +-------------------------------+
                                          |
                                          v
                              +-------------------------------+
                              | 上传过程中自动刷新 STS        |
                              | 或手动暂停 / 继续             |
                              +-------------------------------+
                                          |
                                          v
                              +-------------------------------+
                              | 上传完成后调用                |
                              | POST /file/upload/complete    |
                              +-------------------------------+
                                          |
                                          v
                              +-------------------------------+
                              | 后端校验对象大小              |
                              | 写 repository_pool            |
                              | 写 user_repository            |
                              | 更新 upload_session 状态      |
                              +-------------------------------+
                                          |
                                          v
                              +-------------------------------+
                              | 前端显示上传完成              |
                              +-------------------------------+
```

二、文件预览流程

1. 功能目标

- 支持图片、视频、音频、PDF、文本和其他文件预览

2. 设计思路

二进制媒体文件使用 OSS 临时签名 URL，文本类文件只读取前 4KB 以避免大文件整体加载，其他文件返回下载地址。

3. 流程图

```
+-----------------------+
| 用户点击文件预览       |
+-----------------------+
            |
            v
+-----------------------------------+
| 调用 GET /user/file/preview/:id   |
+-----------------------------------+
            |
            v
+-------------------------------+
| 后端根据扩展名判断文件类型     |
+-------------------------------+
     | 图片/视频/音频/PDF
     |-------------------------------> 生成 OSS 签名 URL
     |
     | 文本
     |-------------------------------> 读取前 4KB 文本
     |
     | 其他
     |-------------------------------> 返回下载地址
            |
            v
+-----------------------+
| 前端弹窗展示预览       |
+-----------------------+
```

三、最近文件流程

1. 功能目标

记录用户最近访问文件，便于快速定位常用内容。

2. 设计思路

后端在文件被成功预览后，将文件 identity 写入 Redis 的 ZSet，score 为最后访问时间戳。

3. 流程图

```
+------------------------+
| 用户成功预览文件       |
+------------------------+
            |
            v
+------------------------------+
| 后端写 Redis ZSet 最近记录    |
+------------------------------+
            |
            v
+----------------------------+
| 调用 GET /user/file/recent  |
+----------------------------+
            |
            v
+------------------------+
| 前端展示最近文件       |
+------------------------+
```

四、搜索 / 筛选 / 排序流程

1. 功能目标

- 按名称搜索
- 按类型筛选
- 按收藏状态筛选
- 按名称、大小、时间排序

2. 设计思路

扩展原有文件列表接口，通过 query、file_type、favorite_only、order_by、order_dir 统一完成搜索、筛选和排序。

3. 流程图

```
+------------------------+
| 用户输入搜索条件        |
| 或切换筛选排序         |
+------------------------+
            |
            v
+----------------------------+
| 调用 GET /user/file/list   |
+----------------------------+
            |
            v
+------------------------------+
| 后端按条件动态构造 SQL       |
| - query                      |
| - file_type                  |
| - favorite_only              |
| - order_by / order_dir       |
+------------------------------+
            |
            v
+------------------------+
| 返回列表结果           |
+------------------------+
            |
            v
+------------------------+
| 前端刷新表格           |
+------------------------+
```

五、收藏流程

1. 功能目标

支持单个收藏和批量收藏。

2. 设计思路

直接在 user_repository 表维护 is_favorite 字段，不额外拆表，便于列表查询时直接筛选。

3. 流程图

```
+-----------------------+
| 用户点击收藏按钮       |
| 或批量收藏            |
+-----------------------+
            |
            v
+-------------------------------+
| 调用收藏接口更新 is_favorite   |
+-------------------------------+
            |
            v
+-----------------------+
| 刷新当前文件列表       |
+-----------------------+
```

六、批量操作流程

1. 功能目标

- 批量收藏
- 批量移动
- 批量删除

2. 设计思路

文件夹的批量操作不能只处理当前节点，而要处理整棵子树；同时移动、删除等操作必须放在事务里保证一致性。

3. 流程图

```
+-----------------------+
| 用户勾选多个文件       |
+-----------------------+
           |
           v
+-----------------------+
| 选择批量操作类型       |
+-----------------------+
    | 批量收藏
    |----------------------------> 更新 is_favorite
    |
    | 批量移动
    |----------------------------> 校验目标目录
    |                               校验不能移动到自己或子目录
    |                               批量更新 parent_id
    |
    | 批量删除
    |----------------------------> 收集整棵子树
                                    批量写 deleted_at
```

七、回收站流程

1. 功能目标

- 删除进入回收站
- 回收站查看
- 恢复
- 彻底删除

2. 设计思路

回收站基于 deleted_at 软删除实现。恢复前会检查父目录状态，避免父目录未恢复但子文件先恢复导致层级错乱。

3. 流程图

```
+------------------------+
| 用户删除文件 / 文件夹  |
+------------------------+
            |
            v
+------------------------+
| 写 deleted_at          |
| 进入回收站             |
+------------------------+
            |
            v
+----------------------------+
| 调用 GET /user/recycle/list |
+----------------------------+
            |
            v
+------------------------+
| 前端展示回收站         |
+------------------------+
      | 恢复                          | 彻底删除
      v                               v
+---------------------------+   +---------------------------+
| 检查父目录是否已恢复      |   | 物理删除数据库记录        |
+---------------------------+   +---------------------------+
      | 是        | 否
      v           v
+-----------+   +------------------------+
| 清空      |   | 拒绝恢复子文件         |
| deleted_at|   +------------------------+
+-----------+
```

八、整体总结

> 本次新增功能采用“前端直传 OSS + 后端发 STS + 资源池去重 + 软删除回收站 + Redis 最近文件”的组合方案，在保证功能完整的同时，也保留了后续继续扩展的空间。

- 上传采用前端直传 OSS，减轻后端压力
- 秒传通过 hash + size 命中资源池实现
- 断点续传依赖 upload_session + checkpoint
- 预览采用“签名 URL + 文本分段读取”组合方案
- 最近文件采用 Redis ZSet 实现
- 收藏采用字段方案实现，轻量直接
- 批量操作与回收站统一基于子树处理和事务控制

文档用途：项目汇报 / 设计说明 / 实现细节说明 / 面试项目讲解